Tabla de contenido

4 Analizadores sintácticos 2
4.1. Introducción 2
¿Qué es un analizador sintáctico (parser)? 2
4.2 Modelo DOM (Document Object Model) 3
Concepto 3
Estructura jerárquica del DOM 3
Cómo funciona DOM 5
4.3 Modelo SAX (Simple API for XML) 8
Concepto 8
DOM vs SAX 8
Cómo funciona 8
Ventajas y desventajas del modelo SAX 9
Cuándo usar SAX 10
4.4 Enlace de datos o Binding (JAXB) 10
Concepto 10
Dependencias Maven 11
Idea general 11
Anotaciones más usadas en Jakarta 13
Ventajas / desventajas 15
Cuándo usarlo 15
4.5 Comparativa de enfoques 16
Anexo: anotaciones Java 16
¿Qué son? 16
¿Cómo funcionan? 17
Anotaciones en Java 17

4 Analizadores sintácticos

4.1. Introducción

En el apartado anterior hemos visto cómo trabajar con ficheros XML como simples archivos de texto.

Sin embargo, para leer, modificar o generar XML de manera estructurada en Java, necesitamos utilizar analizadores sintácticos (parsers).

¿Qué es un analizador sintáctico (parser)?

Un *parser XML* es una herramienta que:

- Lee un documento XML.
- Comprueba que está bien formado (etiquetas bien anidadas, una raíz, etc.).
- Y permite acceder a su contenido desde un programa Java de forma estructurada.

Java proporciona **varios modelos de análisis XML**, que se agrupan en dos grandes familias:

Tipo de parser	**Modelo**	**Características principales**
DOM (Document Object Model)	Basado en árbol	Carga todo el documento XML en memoria y lo representa como un árbol de nodos.
SAX (Simple API for XML)	Basado en eventos	No carga todo el documento. Lo procesa secuencialmente mientras se lee.
JAXB (Java Architecture for XML Binding)	Basado en objetos	Convierte automáticamente entre objetos Java ↔ XML (marshalling/unmarshalling).

Cada enfoque tiene ventajas e inconvenientes, y se usa en distintos escenarios:

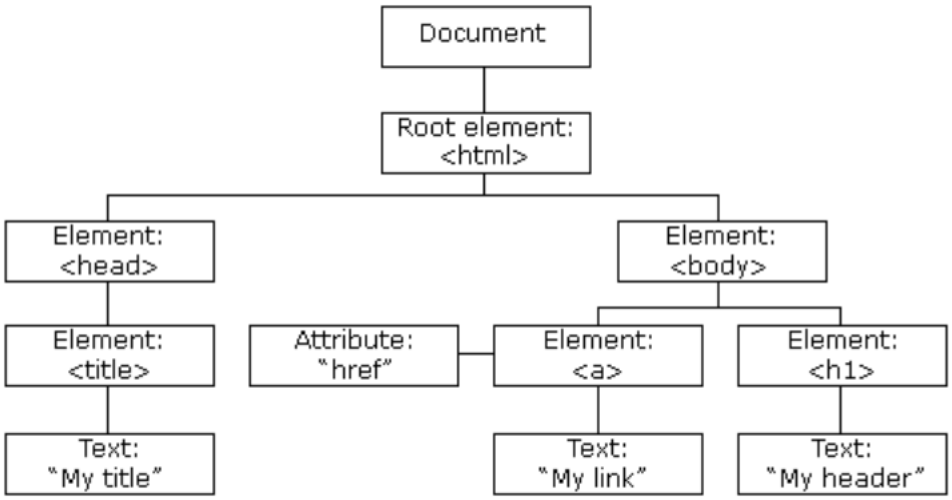
- **DOM** → ideal cuando el XML es pequeño y necesitamos leer y modificar libremente.
- **SAX** → ideal para procesar grandes volúmenes de datos sin ocupar mucha memoria.

- **Binding (JAXB)** → ideal cuando trabajamos con clases Java que deben guardarse o leerse en XML.

4.2 Modelo DOM (Document Object Model)

Concepto

El modelo **DOM (Document Object Model)** representa el documento XML completo como un **árbol de nodos** en memoria (también se aplica a HTML).



Cuando un programa Java usa un parser DOM, **el documento XML se convierte en un árbol de objetos** donde cada **etiqueta, atributo o texto** se convierte en un **nodo** del árbol.

Estructura jerárquica del DOM

Cada documento XML tiene:

1. Un **nodo raíz**, que representa la etiqueta principal (por ejemplo <alumnos>).
2. Dentro, **elementos hijo** (por ejemplo <alumno>).
3. Cada elemento puede tener:
    - **Atributos** (como id="A01"),
    - **Subelementos** (como <nombre> y <edad>),
    - y **nodos de texto** (como Ana o 20)

Lo veremos con el siguiente ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos>
    <alumno id="A01">
        <nombre>Ana</nombre>
        <edad>20</edad>
    </alumno>
    <alumno id="A02">
        <nombre>Luis</nombre>
        <edad>22</edad>
    </alumno>
</alumnos>
```

El parser DOM genera en memoria una estructura **en forma de árbol**:

```
Document
└── Element: alumnos
    ├── Element: alumno (id="A01")
    │   ├── Element: nombre
    │   │   └── Text: "Ana"
    │   └── Element: edad
    │       └── Text: "20"
    └── Element: alumno (id="A02")
        ├── Element: nombre
        │   └── Text: "Luis"
        └── Element: edad
            └── Text: "22"
```

Cada nodo del árbol se representa mediante objetos Java de las clases del paquete `org.w3c.dom (Document, Element, Text, etc.)`.

Cómo funciona DOM

Cuando utilizamos DOM en Java, el proceso de lectura de un XML sigue estas **fases**:

Fase	Descripción	Clases implicadas
Creación del parser	Se obtiene un parser DOM mediante la clase DocumentBuilderFactory, que permite configurar opciones (por ejemplo, validar el XML o ignorar espacios en blanco).	`DocumentBuilderFactory`
Construcción del árbol DOM	El parser lee el fichero XML completo y crea un **objeto Document** en memoria, que representa todo el XML como un árbol jerárquico.	`DocumentBuilder, Document`
Normalización	Se eliminan nodos vacíos y se unifican fragmentos de texto adyacentes.	`doc.normalize()`
Navegación y manipulación	Se pueden buscar nodos, recorrer el árbol, acceder a etiquetas, atributos, textos, añadir o borrar elementos.	`Node, Element, NodeList`
Escritura (opcional)	Si se modifica el árbol, puede escribirse de nuevo a disco para actualizar el XML.	`Transformer, TransformerFactory`

- El parser DOM **lee todo el fichero XML** y **lo carga completamente en memoria**.
- A partir de ahí, el programa puede recorrer el árbol para acceder a los datos.
- Si se realizan cambios, el árbol puede guardarse de nuevo en un fichero XML.

Por tanto:

- Es un modelo **bidireccional** (lectura y escritura).
- Es **muy flexible**, pero consume más memoria y tiempo de carga en XML grandes.

Revisa los ejemplos del aula virtual:

- LeerXML_DOM.java lee el fichero alumnos.xml
- GestionBiblioteca_DOM.java. Ejemplo que lee el contenido de biblioteca,xml, lista los libros, modifica un nodo (año) y atributo (edición). Crea un nuevo libro y escribe en biblioteca_modificada.xml
- GestionBibliotecaCRUD_DOM.java. Versión de CRUD completo sobre biblioteca.xml

Métodos más usados del modelo DOM:

Clase	Método	Descripción
Document	`getDocumentElement()`	Devuelve el elemento raíz del documento.
	`getElementsByTagName(String etiqueta)`	Devuelve todos los elementos con esa etiqueta.
Element	`getTagName()`	Devuelve el nombre de la etiqueta.
	`getAttribute(String nombre)`	Devuelve el valor de un atributo.
	`setAttribute(String nombre, String valor)`	Establece o modifica un atributo.
	`getElementsByTagName(String etiqueta)`	Devuelve los subelementos con esa etiqueta.
	`getTextContent()`	Devuelve el texto contenido en el elemento.
	`setTextContent(String texto)`	Cambia el texto de un elemento.
Node	`getNodeName()`	Devuelve el nombre del nodo.
	`getNodeType()`	Indica el tipo de nodo (elemento, texto, comentario...).
	`getParentNode()`	Devuelve el nodo padre.
	`getChildNodes()`	Devuelve una lista con los nodos hijos.

Clase	Método	Descripción
NodeList	getLength()	Devuelve el número de nodos de la lista.
	item(int index)	Devuelve el nodo en la posición indicada.
DocumentBuilderFactory	setIgnoringComments(boolean)	Ignora comentarios durante el parseo.
	setIgnoringElementContentWhitespace(boolean)	Ignora espacios en blanco entre etiquetas.
Transformer	transform(Source, Result)	Permite escribir o exportar el documento a un fichero XML.

Para los tipos de nodo, getNodeType() devuelve un número constante que indica el tipo de cada nodo. Ejemplo:

Tipo de nodo	Constante	Valor numérico	Ejemplo en XML
ELEMENT_NODE	Node.ELEMENT_NODE	1	<alumno>
ATTRIBUTE_NODE	Node.ATTRIBUTE_NODE	2	id="A01"
TEXT_NODE	Node.TEXT_NODE	3	Ana, 20
COMMENT_NODE	Node.COMMENT_NODE	8	<!-- comentario -->
DOCUMENT_NODE	Node.DOCUMENT_NODE	9	Todo el documento XML

4.3 Modelo SAX (Simple API for XML)

Concepto

El modelo **SAX (Simple API for XML)** es una alternativa a DOM para **leer documentos XML de forma secuencial y basada en eventos**.

A diferencia de DOM, **SAX no carga todo el XML en memoria**, sino que lo analiza **mientras se lee línea a línea**.
Cada vez que el parser encuentra una etiqueta, un texto o un final de elemento, **lanza un evento** que el programa puede interceptar y procesar.

DOM vs SAX

Característica	**DOM**	**SAX**
Tipo de modelo	Árbol en memoria	Lectura secuencial por eventos
Carga en memoria	Todo el documento	Solo el fragmento actual
Acceso a datos	Aleatorio (se puede volver atrás)	Secuencial (solo hacia adelante)
Modificación	Permite modificar y crear XML	Solo lectura (no modifica)
Velocidad y consumo	Más lento, más memoria	Más rápido, menos memoria
Ideal para	XML pequeños o editables	XML grandes o lectura rápida

Cómo funciona

1. Se crea un **parser SAX** usando SAXParserFactory.
2. Se implementa una **clase manejadora** (*handler*) que hereda de DefaultHandler.
3. En esta clase, se sobreescriben los métodos que responden a los **eventos SAX**:
    - startDocument() → Inicio del documento
    - startElement() → Se abre una etiqueta
    - characters() → Se encuentra texto dentro de una etiqueta
    - endElement() → Se cierra una etiqueta

- endDocument() → Fin del documento

4. El parser recorre el XML y va llamando a esos métodos en orden.

Las clases principales (paquete org.xml.sax y javax.xml.parsers) son:

Clase o interfaz	**Descripción**
`SAXParserFactory`	Crea y configura el parser SAX.
`SAXParser`	Objeto parser que procesa el XML y genera eventos.
`DefaultHandler`	Clase base que define los métodos de evento.
`Attributes`	Permite acceder a los atributos de una etiqueta.

Revisa los ejemplos del aula virtual:

- Ejemplo1: LeerXML_SAX.java y ManejadorBiblioteca.java
- Ejemplo2: otro ejemplo que crea el manejador y lee el fichero alumnos.xml
- Ejemplo3: en este caso trabajamos con objetos alumno

Ventajas y desventajas del modelo SAX

Ventajas	**Desventajas**
No carga todo el XML en memoria.	Solo permite lectura secuencial, no se puede modificar.
Rápido y eficiente para documentos grandes.	No se puede retroceder ni acceder libremente a etiquetas anteriores.
Ideal para procesar grandes volúmenes o flujos de datos (logs, importaciones, etc.).	Más difícil de usar para tareas complejas.
No necesita estructuras de datos adicionales.	No genera un árbol ni permite guardar cambios.

Cuándo usar SAX

Escenario	Recomendación
XML grande o streaming (muchos registros)	Ideal
Solo lectura, sin modificar	Ideal
Necesidad de procesar por partes o en tiempo real	Ideal
Necesidad de modificar o guardar XML	Mejor DOM o JAXB (binding)

4.4 Enlace de datos o Binding (JAXB)

Concepto

El *binding* convierte XML directamente en **objetos Java** (*unmarshalling*) y viceversa (**marshalling**), sin recorrer etiquetas manualmente como con DOM/SAX.

El *data binding* XML–Java es el proceso de **mapear automáticamente** un documento XML a **objetos Java** (unmarshalling) y, a la inversa, **generar XML** a partir de objetos Java (marshalling).
Con JAXB no recorres etiquetas ni gestionas nodos: **describes el modelo con anotaciones** y la librería hace la conversión.

Ventajas:

- Código muy corto y legible.
- Mantenimiento sencillo (clases Java anotadas).
- Ideal para ficheros con **estructura estable** y "modelo de dominio" claro.

Limitaciones:

- Menos flexible para XML "irregulares".
- Requiere **clases Java con constructor vacío** y anotaciones

Dependencias Maven

Tenemos dos opciones:

1)Jakarta JAXB (paquete jakarta.xml.bind)

```
<dependencies>
  <dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
  </dependency>
</dependencies>
```

2) javax (JAXB 2.x):

```
<dependencies>
  <dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>2.3.8</version>
  </dependency>
</dependencies>
```

- JAXB venía “dentro del JDK” hasta Java 8 bajo el paquete javax.xml.bind. Desde **Java 11** lo quitaron del JDK (módulos) y **hay que añadirlo como dependencia externa**.
- Además, Oracle donó JAXB a Eclipse Foundation y se **renombró el paquete** a jakarta.xml.bind (Jakarta EE).
- Resultado: hoy conviven dos líneas:
  - **JAXB 2.x (javax)** → javax.xml.bind.*
  - **JAXB 4.x (jakarta)** → jakarta.xml.bind.* (RECOMENDADA EN LA ACTUALIDAD)

No debes mezclar ambas en el mismo proyecto.

Idea general

1. **Definir el modelo de dominio** como clases Java (por ejemplo, Centro, Alumno).
2. **Anotar** esas clases y campos indicando qué es raíz, qué es elemento, qué es atributo, cómo se representa una lista, etc.
3. **Crear un contexto JAXB** sobre ese conjunto de clases.
4. **Unmarshalling**: leer XML → obtener objetos poblados.
5. (Opcional) Manipular los objetos en Java (filtrar, ordenar, añadir...).
6. **Marshalling**: escribir de nuevo los objetos → XML formateado.

Flujo conceptual:

XML
→ [Unmarshalling] →
Objetos Java
→ [Marshalling] →
XML

Requisitos básicos de las clases

- **Constructor sin argumentos** (obligatorio).
- **Getters y setters** (recomendable, salvo que uses FIELD).
- Tipos compatibles o anotar también los tipos compuestos.

Ejemplos aula virtual

- MainLibro.java: Guardar un libro en un fichero XML
- MainCurso.java: En este ejemplo se muestra cómo representar en XML una relación de uno a muchos, es decir, un objeto que contiene una lista de otros objetos. El programa define un curso que incluye varios alumnos.
- MainBiblioteca: En este ejemplo se amplía el uso de JAXB para mostrar cómo manejar **estructuras más complejas**:
  una **biblioteca** compuesta por varias **obras** (objetos), cada una con un **tipo** (enumeración) y un **comentario** (texto simple).

Anotaciones más usadas en Jakarta

Anotación	Dónde se usa	Descripción	Ejemplo en clase Java	Resultado XML
@XmlRootElement(name="...")	En la clase	Indica que la clase representa la **etiqueta raíz** del documento XML. El atributo name define el nombre de esa etiqueta.	java @XmlRootElement(name="alumno") public class Alumno { ... }	<alumno> ... </alumno>
@XmlElement(name="...")	En campos o getters	Mapea un **atributo o propiedad** como **subelemento** dentro del XML. Si no se especifica name, usa el nombre del campo.	java @XmlElement(name="nombre") private String nombre;	<nombre>Lucía</nombre>
@XmlAttribute(name="...")	En campos o getters	Mapea un campo como **atributo XML** (dentro de la etiqueta de apertura).	java @XmlAttribute(name="id") private String id;	<alumno id="A01"> ... </alumno>
@XmlElementWrapper(name="...")	Antes de una lista o colección	Agrupa una colección dentro de una **etiqueta contenedora**. Sin esta anotación, los elementos se escribirían directamente.	java @XmlElementWrapper(name="asignaturas") @XmlElement(name="asignatura") private List<String> asignaturas;	xml <asignaturas> <asignatura>Matemáticas</asignatura> <asignatura>Lengua</asignatura> </asignaturas>
@XmlAccessorType(XmlAccessType.FIELD)	En la clase	Define **cómo accede JAXB a los datos**: por campos (FIELD) o por métodos (PROPERTY). Evita	java @XmlAccessorType(XmlAccessType.FIELD)	JAXB tomará todos los campos de la clase como elementos XML.

Anotación	Dónde se usa	Descripción	Ejemplo en clase Java	Resultado XML
		anotar cada getter si se usa FIELD.	public class Alumno { ... }	
@XmlType(propOrder={...})	En la clase	Especifica el **orden** de las etiquetas dentro del XML.	java @XmlType(propOrder={"nombre","edad"})	xml <alumno><nombre>Lucía</nombre><edad>21</edad></alumno>
@XmlTransient	En un campo o método	Excluye un campo del XML (no se serializa ni se lee).	java @XmlTransient private String password;	El campo password no aparecerá en el XML.
@XmlValue	En un campo	Indica que el valor del objeto se representa como **contenido de texto** (sin subelementos ni atributos).	java @XmlValue private String descripcion;	<comentario>Texto libre</comentario>
@XmlSeeAlso({Clase1.class, Clase2.class})	En la clase base	Indica a JAXB qué otras clases relacionadas debe tener en cuenta (por ejemplo, subclases).	java @XmlSeeAlso({Alumno.class, Profesor.class}) public class Persona { ... }	— (solo afecta al procesamiento interno)
@XmlEnum y **@XmlEnumValue**	En enumeraciones (enum)	Define cómo se deben escribir los valores de una enumeración.	java @XmlEnum public enum Genero { @XmlEnumValue("H") HOMBRE, @XmlEnumValue("M") MUJER }	<genero>H</genero>

Ventajas / desventajas

Ventajas

- **Productividad alta**: muy poco código para leer/escribir XML.
- **Legibilidad**: la estructura del XML se "ve" en las clases (anotaciones).
- **Mantenible**: si cambia el XML, ajustas anotaciones/campos y listo.
- **Integración natural**: trabajas con objetos Java, no con nodos ni eventos.
- **Formateo sencillo**: salida legible con propiedades del marshaller.

Desventajas

- **Menos control fino** que DOM/SAX para XML muy irregulares o con estructuras atípicas.
- **Acoplamiento al modelo**: requiere clases "bien modeladas" y constructor vacío.
- **Curva en casos complejos**: listas anidadas, namespaces múltiples o contenido mixto pueden requerir más anotaciones.
- **Dependencia externa** en Java 11/17+ (no viene en el JDK): debes añadir Jakarta/Javax JAXB.

Cuándo usarlo

- Tienes un **modelo de dominio claro y estable** y quieres importar/exportar datos.
- Necesitas **rapidez de desarrollo** y código limpio (configuración por anotaciones).
- El XML se parece a "objetos" (árbol razonablemente regular).
- Quieres enseñar a los alumnos el **ciclo completo XML ↔ Objetos** sin ruido de parseo.

Mejor evita JAXB si…

- El XML es **enorme** y solo vas a leerlo de forma secuencial (SAX será más eficiente).
- Necesitas **transformaciones o ediciones finas** en el árbol (DOM/XSLT/XPath te dan más control).
- El XML es **muy irregular** o con contenido mixto difícil de mapear a clases.

4.5 Comparativa de enfoques

Criterio	DOM	SAX	JAXB (Binding)
Memoria	Alta (árbol completo)	Baja (streaming)	Media (objetos creados)
Acceso	Aleatorio (navegable)	Secuencial (solo hacia delante)	Por objetos (modelo de dominio)
Edición XML	Sí (crear/modificar nodos)	No	Indirecta (editas objetos y re-escribes)
Complejidad	Media	Media/alta (eventos)	Baja (anotaciones)
Casos ideales	Edición fina, XPath/XSLT	Lecturas masivas, logs	Import/export de datos estructurados

Anexo: anotaciones Java

¿Qué son?

Una **anotación** (annotation) es una **forma de añadir información o metadatos** al código Java.
No cambia directamente lo que el programa hace, pero **sirve para indicar instrucciones, reglas o configuraciones** que pueden ser interpretadas por:

- el **compilador**,
- el **entorno de ejecución (JVM)**, o
- otras **librerías o frameworks** (como Jakarta, Spring, JUnit, etc.).

En el código se escriben precedidas por **@**, por **ejemplo**:

```
@Override
public String toString() {
    return "Texto";
}
```

Aquí @Override es una **anotación estándar de Java** que indica al compilador que estás sobrescribiendo un método de una superclase. Si el nombre del método estuviera mal escrito, el compilador te avisaría.

Las anotaciones existen en Java desde la versión **5 (2004)**, mucho antes de Jakarta. Por tanto, **NO pertenecen a Jakarta**, sino que son una **característica del propio lenguaje Java**. Jakarta, Spring, JUnit, Hibernate, etc. **las utilizan** para

configurar su comportamiento sin necesidad de escribir archivos XML o código extra.

¿Cómo funcionan?

Cuando escribes una anotación, **no ejecuta nada por sí sola**. Quien la interpreta es **otra parte del programa** (una librería o el compilador).

Por ejemplo:

- En **JAXB/Jakarta**, las anotaciones sirven para decir cómo convertir una clase Java a XML.
- En **JUnit**, @Test indica qué métodos son pruebas automáticas.

Anotaciones en Java

Anotaciones básicas del lenguaje (paquete java.lang o java.lang.annotation)

Anotación	**Pertenece a**	**Descripción / Uso principal**
@Override	Java SE	Indica que un método sobrescribe otro de una superclase. El compilador lo verifica.
@Deprecated	Java SE	Marca un elemento (método, clase, campo) como **en desuso**. Muestra advertencia al usarlo.
@SuppressWarnings("...")	Java SE	Indica al compilador que **ignore ciertos avisos** (por ejemplo: unchecked, deprecation).
@SafeVarargs	Java SE	Garantiza que un método con argumentos genéricos de tipo variable es seguro (sin “heap pollution”).
@FunctionalInterface	Java SE	Indica que una interfaz tiene **solo un método abstracto** (usada con expresiones lambda).
@Retention	Java SE	Define **hasta cuándo** se conserva la anotación (SOURCE, CLASS, RUNTIME).

Anotación	Pertenece a	Descripción / Uso principal
@Target	Java SE	Indica **dónde puede aplicarse** una anotación (clase, método, parámetro, campo...).
@Inherited	Java SE	Indica que una anotación se **hereda** por las subclases.
@Documented	Java SE	Indica que la anotación se incluirá en la documentación generada con javadoc.
@Repeatable	Java SE 8+	Permite aplicar la misma anotación varias veces a un mismo elemento.
@Native	Java SE	Indica que una constante puede usarse en código nativo (JNI).

Anotaciones de uso frecuente en frameworks y APIs estándar de Java (no POO pura, pero muy comunes)

Anotación	Pertenece a	Descripción/uso principal
@Test	JUnit	Marca un método como **prueba unitaria**.
@BeforeEach, @AfterEach	JUnit	Ejecuta métodos antes o después de cada test.
@BeforeAll, @AfterAll	JUnit	Ejecuta métodos una vez antes o después de todos los tests.
@DisplayName	JUnit 5	Personaliza el nombre mostrado del test.
@Disabled	JUnit 5	Desactiva temporalmente una prueba.